

QA 관점 GitHub PR 리뷰 수행 방법

PR 체크리스트 · 리뷰어 등록 · 코드 품질 · 테스트 검증 · 머지 전략

GitHub PR

QA 관점

리뷰어 등록 필수

PR 시 팀원분들 리뷰어 등록하는 것 잊지 마세요 ...!!!

강의 목차

QA 관점 GitHub PR 리뷰 수행 방법

1

기본 정보 확인

PR 제목·설명·이슈 연결·변경 규모

2

코드 품질 검토

가독성·네이밍·SOLID·중복 제거

3

테스트 검증

테스트 커버리지·케이스 적절성

4

리뷰어 등록 & 소통

리뷰어 지정·코멘트 작성 원칙

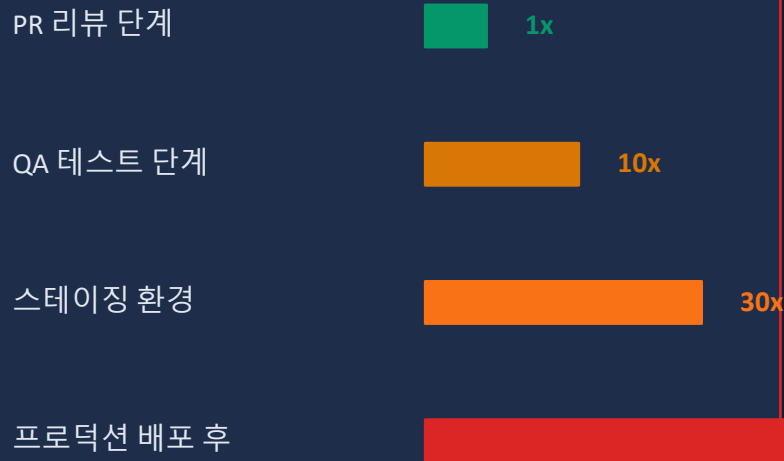
5

머지 전략 & 완료

승인 조건·브랜치 정책·Squash Merge

왜 QA 관점 PR 리뷰가 중요한가?

버그 발견 시점별 수정 비용



→ PR에서 잡으면 가장 저렴하다

버그 조기 발견

코드가 배포되기 전
결함을 잡아 수정 비용 최소화

지식 공유

팀원 간 코드 이해도 향상
버그 팩터 위험 감소

품질 기준 유지

일관된 코딩 컨벤션
아키텍처 방향 정렬

QA 관점 GitHub PR 리뷰 체크리스트

PR 제목

변경 사항을 명확하게 설명하는가?

✅ feat: 로그인 시 JWT 토큰 만료 처리 추가

❌ 수정함

💡 [타입]: [요약] 형식 권장 (feat/fix/refactor/docs)

PR 설명

변경 이유와 목적이 기술되어 있는가?

✅ Why/What/How 구조로 작성

❌ "코드 수정했습니다"

💡 스크린샷, 테스트 결과, 관련 문서 링크 포함

이슈 연결

관련 이슈나 티켓 번호가 연결되어 있는가?

✅ Closes #123 또는 Fixes JIRA-456

❌ 이슈 링크 없음

💡 이슈 자동 닫기: Closes/Fixes/Resolves + 번호

변경 규모

변경된 파일 수와 라인 수가 적절한가?

✅ 핵심 로직 위주, 400라인 이하 권장

❌ 1,000라인+ 단일 PR

💡 크면 기능 단위로 분리 요청 → 리뷰 집중도 ↑

QA 눈으로 코드를 읽는 법

1

가독성 & 네이밍

- ▶ 변수·함수명이 의도를 명확히 표현하는가?
- ▶ 함수 길이는 단일 책임(20~30줄 이하)인가?
- ▶ 매직 넘버·하드코딩 문자열은 없는가?

2

중복 제거 (DRY)

- ▶ 유사 로직이 반복되지 않는가?
- ▶ 공통 유틸 함수로 추출 가능한 부분은?
- ▶ 복사-붙여넣기 코드는 없는가?

3

예외 처리

- ▶ null/undefined 처리가 적절한가?
- ▶ 예외 시나리오가 모두 핸들링되는가?
- ▶ 에러 메시지가 사용자·개발자 친화적인가?

4

보안 취약점

- ▶ 인증·권한 검사가 누락된 엔드포인트?
- ▶ SQL Injection / XSS 가능성?
- ▶ 민감 정보(키·패스워드) 하드코딩?

코드 리뷰 예시

✗ Before

```
def f(x, y, z):  
    d = x * y * z * 0.1  
    if d > 1000:  
        return True  
    return False
```

✓ After

```
DISCOUNT_RATE = 0.1  
PREMIUM_THRESHOLD = 1000  
  
def is_premium_customer(  
    revenue, qty, price  
):  
    discount = revenue*qty*price  
    return discount > PREMIUM_THRESHOLD
```

QA가 반드시 확인해야 할 테스트 포인트

테스트 피라미드

E2E 테스트 10%

통합 테스트 30%

단위 테스트 60%

- ☐ PR에 테스트 코드가 포함됐는가?
- ☐ 기존 테스트가 모두 통과하는가?
- ☐ 새 기능의 엡지 케이스 커버?
- ☐ 커버리지 기준(80%+) 충족?

Happy Path 검증

- ▶ 정상 입력에 대한 기대 출력
- ▶ 기능 흐름 전체 검증
- ▶ 성공 응답 코드 확인

Edge Case 검증

- ▶ 경계값 (0, 최대, 최소)
- ▶ 빈 값·null 입력 처리
- ▶ 대용량 데이터 처리

Failure Path 검증

- ▶ 잘못된 입력값 에러 처리
- ▶ 권한 없는 요청 403 반환
- ▶ 네트워크 오류 복구 로직

효과적인 리뷰어 지정과 코멘트 작성법



PR 시 팀원분들 리뷰어 등록하는 것 잊지 마세요 ...!!!

PR 생성 후 → Reviewers 탭 → 팀원 선택 (최소 1명 이상)

리뷰어 등록 절차

1

PR 생성

GitHub에서 PR 생성 완료

2

Reviewers 탭 클릭

우측 사이드바에서 Reviewers 선택

3

팀원 검색 & 지정

@멘션으로 리뷰어 추가

4

Assignees 본인 지정

본인을 Assignee로 등록

5

슬랙/메신저 공유

PR 링크 팀 채널에 공유

리뷰 코멘트 작성 원칙

✗ 나쁜 코멘트

"이 코드 왜 이렇게 짠 거예요?"

→ 인격 공격, 방향 없음

✓ 좋은 코멘트

"이 로직을 A 방식으로 바꾸면
성능이 $O(n^2) \rightarrow O(n)$ 으로 개선될 것 같아요."

→ 근거 + 대안 제시

💡 질문형 코멘트

"혹시 이 방식을 선택한 이유가 있을까요?
다른 접근도 고려해봤다면 공유해주세요 😊"

→ 협력적 · 개방적

Approve → Merge — 안전하게 끝내는 법

 Merge Commit

기본

전체 이력 보존
복잡한 히스토리

적합: 장기 개발 브랜치 병합

 Squash & Merge

권장

커밋 하나로 압축
깔끔한 main 이력

적합: 기능 단위 PR (권장)

 Rebase & Merge

주의

선형 이력 유지
충돌 위험 높음

적합: 소규모 개인 작업

 머지 전 체크리스트

- | | |
|------------------------|----------------------|
| ✓ 최소 1명 이상 Approve 획득 | ✓ 모든 CI 테스트 GREEN 통과 |
| ✓ Requested Changes 0건 | ✓ 충돌(Conflict) 해결 완료 |
| ✓ 리뷰 코멘트 전부 Resolve | ✓ 브랜치 최신 상태 (sync) |

 브랜치 네이밍 규칙

feat/로그인-JWT-토큰

신규 기능

fix/결제-금액-오류

버그 수정

refactor/사용자-서비스

리팩토링

docs/README-업데이트

문서 수정

QA 관점 PR 리뷰 완전 체크리스트

PR 리뷰 전 아래 항목을 모두 확인하세요

기본 정보

- ☐ PR 제목이 변경 사항을 명확히 설명하는가?
- ☐ PR 설명에 Why/What/How가 기술됐는가?
- ☐ 관련 이슈/티켓 번호가 연결됐는가?
- ☐ 변경 파일 수와 라인 수가 적절한가? (400줄↓)
- ☐ 팀원 리뷰어가 등록됐는가? (필수!)

코드 품질

- ☐ 네이밍이 의도를 명확히 표현하는가?
- ☐ 중복 코드가 없는가? (DRY)
- ☐ 매직 넘버·하드코딩이 없는가?
- ☐ 예외 처리가 모든 케이스를 커버하는가?
- ☐ 보안 취약점(인증·SQL·XSS)은 없는가?

테스트

- ☐ 테스트 코드가 PR에 포함됐는가?
- ☐ 기존 테스트가 모두 통과하는가?
- ☐ Happy/Edge/Failure Path 모두 커버?
- ☐ 코드 커버리지 기준(80%+)을 충족하는가?
- ☐ CI 파이프라인이 Green인가?



PR 시 팀원분들 리뷰어 등록하는 것 잊지 마세요 ...!!! → Reviewers 탭 필수 지정 | 최소 Approve 1건 후 Merge

핵심 요약

QA 관점 GitHub PR 리뷰 수행 방법

01

기본 정보부터 확인

PR 제목·설명·이슈 연결·400라인 이하 변경 규모 준수

02

코드 품질은 QA의 눈으로

가독성·중복 제거·예외 처리·보안 취약점 검토

03

테스트 없으면 Merge 없다

Happy/Edge/Failure Path, CI Green, 커버리지 80%+ 확인

04

리뷰어 등록은 필수!

PR 생성 즉시 팀원 Reviewer 지정 → 협력적 코멘트 작성

05

Squash Merge로 깔끔하게

Approve 1건↑ + CI Green + Conflict 0 → Squash & Merge



좋은 PR 리뷰 = 팀의 코드 품질 = 제품의 안정성 | QA는 버그 발견자가 아닌 품질 설계자입니다